Agentomics: an agentic system that autonomously develops novel state-of-the-art solutions for biomedical machine learning tasks

Vlastimil Martinek, Andrea Gariboldi, Dimosthenis Tzimotoudis, Mark Galea, Elissavet Zacharopoulou, Aitor Alberdi Escudero, Edward Blake, David Čechák, Luke Cassar, Alessandro Balestrucci, Panagiotis Alexiou

Downloaded from https://academic.oup.com/bioinformatics/article/42/Supplement_1/btag250/8726289 by University of Malta user on 28 July 2026

Bioinformatics, 2026, **42**, btag250
https://doi.org/10.1093/bioinformatics/btag250
ISMB proceedings Supplement

OXFORD

Agentomics: an agentic system that autonomously develops novel state-of-the-art solutions for biomedical machine learning tasks

Vlastimil Martinek[1,2], Andrea Gariboldi[1,2], Dimosthenis Tzimotoudis[1,2], Mark Galea[1,2], Elissavet Zacharopoulou[1,2], Aitor Alberdi Escudero[1,2], Edward Blake[1,2], David Čechák[3,4], Luke Cassar[1,2], Alessandro Balestrucci[1,2], Panagiotis Alexiou[1,2,*]

[1]Centre for Molecular Medicine and Biobanking, University of Malta, Msida, MSD 2080, Malta
[2]Department of Applied Biomedical Science, Faculty of Health Sciences, Msida, MSD 2080, Malta
[3]Central European Institute of Technology, Masaryk University, Brno, 625 00, Czech Republic
[4]National Centre for Biomolecular Research, Faculty of Science, Masaryk University, Brno, 625 00, Czech Republic
*Corresponding author. Centre for Molecular Medicine and Biobanking, University of Malta, Msida, MSD 2080, Malta. E-mail: panagiotis.alexiou@um.edu.mt.

Abstract

Motivation: **Extracting knowledge from biomedical data is crucial for advancing our understanding of biological systems and developing novel therapeutics. The quantity, quality, and resolution of biomedical data constantly evolves, requiring the automation of biomedical machine learning (ML). Existing Automated ML tools lack flexibility, while large language models (LLMs) struggle to consistently deliver reproducible machine learning codebases, and existing LLM Agent-powered solutions lag behind human-engineered ML models.**

Results: **Here, we introduce Agentomics, an autonomous LLM-powered agentic system for end-to-end ML experimentation. Given a biomedical dataset, Agentomics implements various ML modeling strategies, and produces a ready-to-use ML model. Agentomics introduces strict validation checkpoints for standard ML development steps, allowing gradual development on top of working code with defined interfaces and validated artifacts. Further, it offers native support for biomedical foundation models that can be leveraged during experimentation. The generic nature of Agentomics allows the user to create ML solutions for a large variety of datasets and use various LLMs. We evaluate Agentomics across 20 datasets from the domains of Protein Engineering, Drug Discovery, and Regulatory Genomics. When benchmarked against other agentic systems, Agentomics outperformed them in all tested domains. When benchmarked against human expert solutions, Agentomics generated novel state-of-the-art models for 11/20 established benchmark datasets.**

Availability and implementation: **Agentomics is implemented in Python. Source code and documentation are freely available at: https://github.com/BioGeMT/Agentomics-ML**

1 Introduction

The past decades have witnessed the transformation of molecular biology into a truly data-driven science, in large part due to the growth in the quantity and variety of molecular biology data generated by technologies such as mass spectrometry and high-throughput sequencing (Marx 2013, Stephens *et al.* 2015).

An important step in the analysis workflow for biomedical data is the use of Machine Learning (ML) techniques. Automated Machine Learning (AutoML) frameworks, such as Auto-sklearn (Feurer *et al.* 2015), AutoGluon (Erickson *et al.* 2020), and H2O (LeDell *et al.* 2020), can automate the model selection and hyperparameter optimization steps of the ML development process. These frameworks are reliant on pre-defined data representations and limited architecture search (Azevedo *et al.* 2024). Thus, their utility is limited when applied to biomedical data, which is sometimes best represented in non-standard domain-aware representations. Additionally, new data modalities are often introduced, and slowly standardized. Accurate predictive models for biomedical data may require custom deep neural network architectures or the use of foundation models.

The capability of large language models (LLMs) to generate code is already assisting general software engineering tasks. For ML experimentation, LLM-generated code can be manually executed in a run environment and incorporated into the ML pipeline. While this approach enables the development of customized data representations, as well as task-specific model architectures, it also introduces important limitations. In particular, the absence of an error-

© The Author(s) 2026. Published by Oxford University Press.
This is an Open Access article distributed under the terms of the Creative Commons Attribution License (https://creativecommons.org/licenses/by/4.0/), which permits unrestricted reuse, distribution, and reproduction in any medium, provided the original work is properly cited.

Downloaded from https://academic.oup.com/bioinformatics/article/42/Supplement_1/btag250/8726289 by University of Malta user on 28 July 2026

correction mechanism often results in low success rates for generated code, requiring human oversight for refinement and debugging (Jimenez *et al.* 2024).

LLMs can be extended with tools that enable them to interact with an environment, and perform tasks such as code execution. Such implementations have come to be referred to as LLM Agents (Wang *et al.* 2024). Multi-step LLM Agents can also receive and react to error messages by adjusting their tool calls, effectively creating a loop that can be utilized for software development.

Recently, several specialized LLM Agents have been introduced that are able to autonomously develop ML models. These were categorized (Miller *et al.* 2025) as 'generalist' ML coding agents (AIDE, Jiang *et al.* 2025; MLAgentBench, Huang *et al.* 2024), or more specialized 'biomedical' ML coding agents (STELLA, Jin *et al.* 2025; Biomni, Huang *et al.* 2025).

AIDE (Jiang *et al.* 2025) uses a tree-search methodology, treating each potential ML solution as a node. It iteratively refines promising candidates, achieving top performance among Agent frameworks on benchmarks such as MLE-Bench (Chan *et al.* 2024).

MLAgentBench (Huang *et al.* 2024) uses the ReAct (Yao *et al.* 2023) and Reflexion (Shinn *et al.* 2023) prompting frameworks to guide an LLM-based Agent to construct ML solutions for problems from diverse domains (text, images, graphs, and tabular data).

STELLA (Jin *et al.* 2025) is a multi-agent system composed of manager, developer, critic, and tool creation agents. It improves results over time by gathering successful strategies in a template library. STELLA reports outperforming LLM methods and other Agents on various biomedical research tasks.

Biomni (Huang *et al.* 2025) is a general-purpose biomedical Agent that uses an expert-vetted environment of pre-installed tools to assist with biomedical tasks, and retrieves tools from this set based on user goals. It beats various baselines on biomedical knowledge and reasoning, and demonstrates the capability to create multi-omics analysis workflows.

Despite their distinct approaches, LLM Agents have largely underperformed compared to human experts on a variety of biomedical ML benchmarks (Miller *et al.* 2025).

Beyond performance, LLM Agents that can autonomously execute code pose a security risk to the environment in which they operate. Implementing security measures to constrain LLM Agents adds overhead to the successful use of such methods. A lack of such measures can have catastrophic consequences, such as data loss due to the execution of malicious code on the host machine.

Biomedical datasets are often sensitive and may be legally restricted in sharing or publishing. Support for local and open-source LLMs, that do not require API calls to a provider's server, is important in order to securely use LLM Agents for ML on real biomedical data.

In this study, we present Agentomics, an autonomous LLM Agent for automating biomedical end-to-end ML experimentation. When benchmarked against other agentic systems, Agentomics outperformed them in all tested domains. When benchmarked against human expert solutions, Agentomics generated novel state-of-the-art models for 11/20 established benchmark datasets across the biomedical domains of Protein Engineering, Drug Discovery, and Regulatory Genomics.

Agentomics uses an iterative process to explore various ML strategies, while being able to use domain-specific foundation models to enhance its modeling capabilities. It also supports a wide variety of biomedical data modalities including, but not limited to, DNA, RNA, protein, and small molecule sequences. A single Agentomics run produces all necessary artifacts needed for running inference and retraining models with new data. Additionally, it produces a report summarizing the final model and its predictive performance. All runs are fully containerized in a secure environment and prevent test set leakage and metric hallucinations. Agentomics supports a large selection of LLM providers and models, including local LLMs that can be used to securely retain sensitive biomedical data on the host machine.

2 Materials and methods

Agentomics is an autonomous system based on LLM Agents for end-to-end ML experimentation. Agentomics has been designed to produce ML models for classification and regression tasks. It ingests a single dataset, experiments with multiple strategies, and outputs a single ML model and all necessary artifacts for using it.

2.1 Interface

The inputs of Agentomics are: (i) a training dataset file that will be available to agents, (ii) an optional dataset description file that will be injected into the agent's base prompt, (iii) an optional test dataset file that will be used at the end of the run to report metrics to the user, and (iv) an optional validation dataset file for early stopping and estimating performance, which, if not provided, is created by Agentomics by splitting the training dataset. All dataset files are required to be in a CSV format and contain a label column.

The outputs are: (i) scripts with a unified interface and the final model artifacts necessary for inference and re-training, (ii) a conda environment necessary to run the produced scripts, (iii) a PDF report including the summary of the chosen ML approach and train, test, and validation metrics, and (iv) any miscellaneous files (e.g. exploratory scripts and results) that the LLM Agent created during the run.

Among the outputs, a successful run produces two scripts with a unified interface. The training script can be used to retrain the ML model, and always takes as input (i) a train data file in the same format as the original training file, and (ii) a validation data file. It produces a new folder with all necessary artifacts for inference. These artifacts can then be passed to the second script, the inference script, to predict labels of new data. This takes as input a single file in the same format as the original training data, minus labels. It produces an output file with predicted labels for all samples in this input file.

The run can be further customized by various parameters, including the maximum time duration of the run, LLM backbone, and validation metric to use for optimization. Additionally, the user can set a split budget, specifying for how long a run is allowed to re-split data into new train and validation sets, and a baseline budget, specifying how many initial iterations should implement only simple baseline ML models.

2.2 Security and privacy

Agentomics operates within an isolated Docker container with read-only mounts. This prevents any changes to the user's host system outside of this Docker container. Agentomics optionally supports deployment with locally hosted LLMs. This capability addresses privacy concerns in biomedical datasets.

Downloaded from https://academic.oup.com/bioinformatics/article/42/Supplement_1/btag250/8726289 by University of Malta user on 28 July 2026

2.3 Tools

Agentomics uses six tools to enable agents to interact with their execution environment, which contains a pre-installed conda environment with packages like scikit-learn, torch, and transformers.

1) The bash tool enables running arbitrary bash commands and installing packages into the agent's conda environment. It returns the console output of the command. This tool can create, edit, and remove files in the workspace.
2) The python write tool checks the syntax of a python file content and writes it into the Agentomics workspace. It returns whether the writing was successful. This tool can create files in the workspace.
3) The python run tool executes a python file with provided keyword arguments and returns the console output of the script and its run duration. This tool can create, edit, and remove files in the workspace.
4) The edit tool replaces specific content in a file and allows editing existing files with low token usage. It returns how many edits were done.
5) The foundation models info tool either returns a summary of all the available foundation models or detailed information and code snippets of a specific available foundation model.
6) The final result tool is called by the agent at the completion of each step to return the step's structured output in JSON format.

The output of each tool is capped at 5000 characters to prevent flooding of the LLM context. All tools can return error codes, messages, and error stack traces to the agent's context.

2.4 Foundation models

Agentomics runs have access to several pre-downloaded foundation models from various biomedical domains. Six variants of ESM-2 (Lin *et al.* 2023) for protein sequences, five variants of HyenaDNA (Nguyen *et al.* 2023) and nine variants of NucleotideTransformer (Dalla-Torre *et al.* 2025) for DNA sequences, three variants of RiNALMo (Penić *et al.* 2025) for RNA sequences, and seven variants of ChemBERTa (Ahmad *et al.* 2022) and one variant of MolFormerXL (Ross *et al.* 2022) for small molecules.

An Agentomics run can retrieve the documentation and example code snippets of the available foundation models through the `foundation models info tool`. This allows generated python scripts to quickly implement architectures leveraging pre-trained foundation models. The list of available foundation models is easily extensible through a configuration file, allowing the addition of Hugging Face (Wolf *et al.* 2020) models.

2.5 Predefined steps

Agentomics follows a sequence of predefined steps (Fig. 1). Each predefined step results in a structured output that serves as a concise summary for subsequent steps, reducing the overall context usage. Each step is implemented by a separate LLM Agent with a clean context that is prompted with (i) base prompt containing guidelines, (ii) structured outputs of all previous steps, and (iii) step-specific instructions. Each LLM Agent can use any of the tools from the Tools section and may use them repeatedly to interact with a shared environment (e.g. explore data, create files, execute code, observe results). Eventually, the Agent must submit the step-structured output through the `final result tool`. All tool call messages are kept in the Agent's context until the step is completed and the final step result has successfully passed validation.

Output must pass structured validation (valid JSON) and step-specific functional validation (Fig. 1) before being considered completed. If validation fails, the LLM Agent receives error messages and must continue implementing the same step and re-submit the step output. This validation-retry loop continues until the validation passes, guaranteeing that subsequent steps operate on validated outputs and artifacts. If an Agent exceeds the retry limit, Agentomics restarts from the first step.

The steps can be summarized as follows:

1) *Data Exploration*: task is to explore data and output descriptive statistics, feature analysis, and domain-specific insights.
2) *Data Splitting*: task is to split the input data into training and validation sets and provide their paths. The following are programmatically validated: (i) split files existence and naming (ii) retaining correct column names (iii) non-overlapping samples between the splits. This step is skipped if the user provided a validation dataset.
3) *Data Representation*: task is to output text description of the intended data representation approach.
4) *Model Architecture*: task is to output text description of the intended model architecture and hyperparameters.

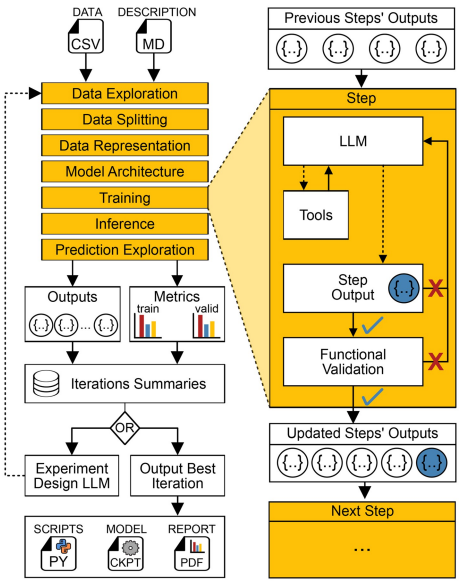


Figure 1 Architecture of Agentomics. Using input data and a description, Agentomics sequentially completes predefined ML experimentation steps using LLM Agents. Each step receives the outputs of previous steps, and is structurally and functionally validated, and potentially retried, before being considered successful. Once all steps are completed, their outputs and model metrics are saved as an Iteration Summary. The Experiment Design LLM then uses Iteration Summaries from all completed iterations to create instructions for each step of the next iteration. Once a run ends, Agentomics outputs files of the best iteration (based on the validation score), including training and inference scripts, model files, and a report including metrics and summarizing the implemented strategy. Dotted arrows represent LLM-dependent actions.

Downloaded from https://academic.oup.com/bioinformatics/article/42/Supplement_1/btag250/8726289 by University of Malta user on 28 July 2026

5) *Training*: task is to implement a training script and output its path, train a model using the split data, output a folder path containing training artifacts, and output a text summary of the implementation. The following are programmatically validated: (i) training script existence, accepted parameters, and naming (ii) training artifacts existence, and naming (iii) training script runability using a different subset of data, and production of the same artifacts as the submitted trained model.
6) *Inference*: the task is to implement an inference script, and output its path and a text summary of the implementation. The following are programmatically validated: (i) Inference script existence, accepted parameters, and naming (ii) inference script runability using a different subset of data, and production of predictions for all samples in a correct format.
7) *Prediction Exploration*: task is to output statistics and insights from exploring model validation predictions and identifying potential prediction biases.

Once this last step is completed, the produced training artifacts and inference script are used to programmatically run inference on the validation and train splits. The predictions are then used to compute various metrics (Accuracy, AUROC, etc) for both splits. This allows an objective comparison of training and validation metrics for different strategies without relying on LLM-reported metrics. These seven steps are defined as an *iteration* of the run. Each successful iteration produces a single working ML model.

2.6 Iterative experimentation and model selection

At the end of each iteration, a model is produced (M in Line 6, Algorithm 1), which we use to generate predictions and obtain training and validation metrics (Lines 12–13, Algorithm 1). These, together with structured outputs of all seven steps (s in Line 6, Algorithm 1), create an iteration summary (Line 18, Algorithm 1).

At the beginning of a new iteration, we prompt the experiment design LLM to generate detailed instructions for each step (Line 4, Algorithm 1) that are used to extend the base prompt of that iteration (Line 5, Algorithm 1). The prompt for the experiment design LLM contains the iterations' summaries of all completed iterations (Line 18, Algorithm 1), enabling it to make connections between each iteration's ML strategy and its performance. Additionally, it contains the dataset description, as well as information about the available GPU and CPU resources, remaining runtime, name of the validation metric to optimize, and information about available tools and foundation models. If configured, it also specifies whether the next iteration is allowed to split or re-split the data into training and validation sets, and whether the architectures should be limited to baseline models. In the prompt, we also include guidelines to reduce cases of agents over-optimizing one strategy through incremental changes and instead focus on a more exploratory approach. If available for the given LLM backbone, this experiment design LLM call is set to high-reasoning effort.

An Agentomics run uses a time and/or an iteration limit. All iterations have access to the same file system and can repurpose models and scripts from previous iterations. After this limit is reached the run ends, and the final output falls back to the completed iteration with the best validation metric score (output iteration) (Line 20, Algorithm 1). Only iterations using the latest train-validation split are compared against each other to preserve fair comparisons (Lines 7–11, Algorithm 1). The specific validation metric to use for determining the best iteration can be adjusted by the user. If a test set is provided with the dataset, files and scripts from this iteration are then used to programmatically run inference and compute test metrics (Line 20, Algorithm 1). Note that the test set is completely isolated from any of the LLM Agents at all times in order to prevent any exploitation or data leakage.

2.7 Evaluation

To evaluate Agentomics in biomedical ML experimentation, we use a set of standard and established benchmarks with existing state-of-the-art models, namely ProteinGym (Notin *et al.* 2023) for Protein Engineering tasks ($N = 6$), Polaris Hub (Ash *et al.* 2025) for Drug Discovery tasks ($N = 9$), and Genomic Benchmarks (Grešová *et al.* 2023) and miRBench (Sammut *et al.* 2025) for Regulatory Genomics tasks ($N = 5$).

For Protein Engineering and Drug Discovery tasks, we follow the BioML-bench framework (Miller *et al.* 2025), which enables us to compare against various autonomous LLM agents, as well as a leaderboard of expert human-engineered solutions (Tables 1 and 2).

For Regulatory Genomics tasks, we compare against Biomni, the strongest overall agent as reported by BioML-bench (Miller *et al.* 2025), using the same LLM backbone. We also compare against a leaderboard of expert human-engineered models, including the current state-of-the-art (Qiao *et al.* 2024, Thoutam and Ellsworth 2024, Sammut *et al.* 2025, Yu *et al.* 2025, Zurgham and Mehwish 2025). The leaderboard has been created by manually curating all published manuscripts that referenced the dataset in use, and reported test set metrics. The leaderboard is available as part of the Agentomics code repository.

Algorithm 1 Agentomics Iterative Experimentation

Require: Dataset D, test set D_{test}, prompt p, iterations t
1: **Initialize:** $M_{\text{best}} \leftarrow$ None, $m_{\text{best}} \leftarrow$ None
2: **Initialize:** $D_{\text{val}} \leftarrow$ None, $S \leftarrow \emptyset$
3: **for** $i = 1$ to t **do**
4: $e \leftarrow \text{EXPERIMENTDESIGN}(S)$ ▷ Create instructions
5: $p_i \leftarrow p \oplus e$ ▷ Create iteration prompt
6: $M, s, D_{\text{train}}, D_{\text{val},i} \leftarrow \text{STEPS}(D, p_i)$
▷ Create model, step summaries, and splits
7: **if** $D_{\text{val},i} \neq D_{\text{val}}$ **then** ▷ Check if split changed
8: $M_{\text{best}} \leftarrow$ None ▷ Reset best model
9: $m_{\text{best}} \leftarrow$ None ▷ Reset best metrics
10: $D_{\text{val}} \leftarrow D_{\text{val},i}$ ▷ Update the current split
11: **end if**
12: $m_{\text{train}} \leftarrow \text{EVALUATE}(M, D_{\text{train}})$ ▷ Train metrics
13: $m_{\text{val}} \leftarrow \text{EVALUATE}(M, D_{\text{val}})$ ▷ Validation metrics
14: **if** m_{val} improves m_{best} **then**
15: $M_{\text{best}} \leftarrow M$ ▷ Update best model
16: $m_{\text{best}} \leftarrow m_{\text{val}}$ ▷ Update best metrics
17: **end if**
18: $S \leftarrow S \oplus (m_{\text{train}}, m_{\text{val}}, s, i)$ ▷ Update summaries
19: **end for**
20: $m_{\text{test}} \leftarrow \text{EVALUATE}(M_{\text{best}}, D_{\text{test}})$ ▷ Test set evaluation
21: **return** $M_{\text{best}}, m_{\text{test}}$

Downloaded from https://academic.oup.com/bioinformatics/article/42/Supplement_1/btag250/8726289 by University of Malta user on 28 July 2026

Table 1 Mean leaderboard placement ± SEM of LLM agents against a leaderboard of expert human-engineered models.[a]

Domain	Agent name	Leaderboard (%)	Completion (%)
Protein Engineering	Agentomics	**75.92 ± 8.00**	**100.0**
	AIDE	35.99 ± 10.96	75.0
	MLAgentBench	13.52 ± 9.18	**100.0**
	STELLA	15.41 ± 6.81	87.5
	Biomni	29.93 ± 11.04	**100.0**
	Zero-Shot	6.37 ± 4.51	55.6
Drug Discovery	Agentomics	**34.29 ± 5.33**	**100.0**
	AIDE	31.06 ± 8.88	80.6
	MLAgentBench	22.45 ± 5.93	**100.0**
	STELLA	12.34 ± 6.62	91.7
	Biomni	32.55 ± 8.61	**100.0**
	Zero-Shot	10.45 ± 3.39	48.1
Regulatory Genomics	Agentomics	**60.15 ± 10.94**	**100.0**
	Biomni (Codex)	30.56 ± 10.31	**100.0**
	Zero-Shot	21.53 ± 8.85	80.0

[a] Completion % represents the proportion of runs that successfully produced predictions. Leaderboard % is calculated only from successful runs.

For each dataset, we run Agentomics and generate an ML model, followed by computing metrics using a held-out test set defined by that benchmark dataset. For Protein Engineering and Drug Discovery datasets, we use the BioML-bench framework to run Agentomics and follow the exact same evaluation criteria and time restrictions as competing agents (Miller *et al.* 2025). Agents are compared based on the percent of human leaderboard solutions they outperform. We also compare against a Zero-Shot LLM baseline that generates a solution in a single response.

For each dataset, we run three replicates for a duration of 8 h each, on a Linux machine with access to NVIDIA RTX A4000 GPU and 48 CPU cores. For all experiments, we ensure the Agents do not have access to the test portions of datasets.

We used OpenAI GPT-5.1-Codex-Max as the LLM backbone for all Agentomics, Zero-Shot, and Regulatory Genomics Biomni runs. The split budget, during which Agentomics is allowed to re-split train and validation data, was set to half of the run (4 h). The baseline budget, during which Agentomics is prompted to use only simple architectures and data representations to set a baseline, was set to four iterations. The validation metrics were dependent on the dataset, based on which metric is reported by state-of-the-art models or BioML-bench benchmarks. Classification datasets used either Accuracy (ACC), area under the precision-recall curve (AUPRC), or area under the ROC curve (AUROC). Regression datasets used either mean average error (MAE), Pearson correlation coefficient (r_p), or Spearman correlation coefficient (r_s).

A complete trace of all Agent actions and code produced during the run is exported. We calculate train, validation, and test metrics, as well as money spent on LLM API calls for each run. We also measure success rate: the percentage of runs that (i) produce predictions, (ii) produce a valid conda environment, (iii) produce an inference script that can be used to infer arbitrary data of the same structure, and (iv) produce a training script that can be used to re-train the model with arbitrary data of the same structure.

3 Results

3.1 Agentomics outperforms other agents in all domains

In order to evaluate the performance of Agentomics against other LLM Agents, we evaluated our method based on the percentage of human leaderboard solutions it outperformed, following the methodology of the BioML-bench (Miller *et al.* 2025) framework.

We performed triplicate runs on each dataset, and compared directly against leaderboard percentile of successful runs of other LLM Agentic methods as reported by BioML-bench (Table 1) or our experiments.

Agentomics achieved mean percentiles of 75.92, 34.29, and 60.15 for Protein Engineering, Drug Discovery, and Regulatory Genomics, respectively, outperforming all other benchmarked methods in all domains.

3.2 Agentomics produces new state-of-the-art for the majority of tested datasets

We have evaluated the best replicate (out of 3) for Agentomics against the reported human state-of-the-art on each of the 20 benchmark datasets. For Protein Engineering tasks, Agentomics outperformed human state-of-the-art in all 6/6 datasets. In Drug Discovery tasks, it achieved new state-of-the-art in 2/9 datasets. In Regulatory Genomics, it matched or improved on the current human state-of-the-art in 4/5 datasets (Table 2). Overall, Agentomics produced new state-of-the-art in 19/60 runs.

3.3 Agentomics indirectly optimizes for generalization

We theorized that optimizing validation performance across iterations may result in indirect overfitting to the validation data and thus hurt generalization. To estimate if Agentomics optimizes generalization, we computed the Pearson correlation coefficient of the directly optimized validation score against the hidden test score over all iterations of all runs (Fig. 2A). We observed high validation-test score correlations in the Regulatory Genomics runs (mean 0.986 ± 0.026), and slightly lower correlations in the Protein Engineering runs (mean 0.746 ± 0.226), and Drug Discovery runs (mean 0.807 ± 0.286). Validation-test score correlation across all runs was generally high (Fig. 2) with a median value of 0.964. This suggests that optimizing validation performance generally does not overfit to the validation data, and translates well to unseen test set performance. This finding highlights the ability of Agentomics to autonomously split train-validation data in a representative manner that optimizes well for generalization.

3.4 Agentomics discovers better-performing ML strategies over time

A single Agentomics run implements various ML strategies across many iterations. If an iteration's submitted model achieves a new best validation score, that iteration is set to be the output iteration and will be the result of the finished run. We computed the validation and test scores for a would-be output iteration across the

Downloaded from https://academic.oup.com/bioinformatics/article/42/Supplement_1/btag250/8726289 by University of Malta user on 28 July 2026

Table 2 Agentomics mean ± STD, worst-of-3, and best-of-3 models compared to human state-of-the-art (SoTA) models across benchmark datasets.[a]

Dataset	Prediction task	Metric	Agentomics worst of 3	Agentomics mean	Agentomics best of 3	Human SoTA
Human Enhancers Cohn ($N = 27\,791$)	Experimental human enhancers versus size-matched random genomic regions.	ACC	0.738	0.750 ± 0.011	**0.759**	0.747
Human Enhancers Ensembl ($N = 154\,842$)	Human enhancer elements against random genomic background sequences.	ACC	0.880	0.897 ± 0.018	0.916	**0.933**
Human OCR Ensembl ($N = 174\,756$)	Open chromatin regions against size-matched random genomic sequences.	ACC	0.754	0.790 ± 0.035	**0.825**	**0.825**
Drosophila Enhancers Stark ($N = 6914$)	Drosophila enhancer sequences against random genomic controls.	ACC	0.803	0.819 ± 0.018	**0.838**	0.686
AGO2 CLASH Hejret2023 ($N = 8193$)	MicroRNA–target interactions.	AUPRC	0.774	0.832 ± 0.054	**0.880**	0.860
SPIKE-SARS2-Starr-2020 ($N = 3803$)	ACE2 binding fitness for SARS-CoV-2 spike RBD single-site variants.	r_s	0.247	0.559 ± 0.348	**0.934**	0.747
SPA-STAAU-Tsuboyama-2023 ($N = 1036$)	Stability of *Staphylococcus aureus* protein A multi-substitution variants.	r_s	0.585	0.848 ± 0.227	**0.986**	0.796
PSAE-PICP2-indels ($N = 1220$)	Stability effects of indel variants in photosystem I subunit IV.	r_s	0.790	0.847 ± 0.093	**0.954**	0.847
CBX4-HUMAN-multi-sub ($N = 918$)	Stability of human CBX4 SUMO ligase multi-substitution variants.	r_s	0.678	0.816 ± 0.153	**0.981**	0.848
Q8EG35-SHEON-indels ($N = 332$)	Organismal fitness for *Shewanella* MtrA protein indel variants.	r_s	0.894	0.927 ± 0.029	**0.949**	0.775
CSN4-MOUSE-indels ($N = 197$)	Stability of mouse COP9 signalosome subunit 4 indel variants.	r_s	0.900	0.946 ± 0.041	**0.979**	0.906
pkis2-egfr-wt-c-1 ($N = 496$)	EGFR wild-type kinase inhibitors.	AUPRC	0.692	0.727 ± 0.031	0.752	**0.866**
adme-fang-hclint-1 ($N = 2229$)	Human liver microsomal intrinsic clearance from SMILES.	r_p	0.679	0.688 ± 0.009	0.697	**0.778**
adme-fang-hppb-1 ($N = 126$)	Fraction unbound in human plasma protein binding.	r_p	0.709	0.757 ± 0.053	0.815	**0.888**
adme-fang-solu-1 ($N = 1578$)	Compound solubility from SMILES using standardized ADME protocols.	r_p	0.599	0.612 ± 0.015	0.628	**0.770**
cyp2d6-substrate ($N = 532$)	CYP2D6 enzyme substrates for drug–drug interaction risk.	AUPRC	0.655	0.674 ± 0.018	0.690	**0.724**
lipophilicity ($N = 3360$)	Lipophilicity (logP) from SMILES.	MAE	0.548	0.534 ± 0.016	0.516	**0.392**
herg ($N = 523$)	hERG potassium channel blockers to assess cardiotoxicity risk.	AUROC	0.823	0.854 ± 0.035	**0.892**	0.891
bbb-martins ($N = 1624$)	Blood–brain barrier penetration.	AUROC	0.863	0.879 ± 0.015	0.892	**0.933**
caco2-wang ($N = 728$)	Intestinal permeability in Caco-2 assay.	MAE	0.368	0.310 ± 0.051	**0.277**	0.282

[a] Descriptions of Protein Engineering and Drug Discovery datasets are adapted from BioML-bench (Miller *et al.* 2025).
Bold values denote best metrics on the dataset.

duration of a run, normalized them per run (0 meaning lowest score from any iteration, 1 meaning highest score from any iteration), and plotted their averages stratified by domain (Fig. 2B and C).

In all domains, we observe that Agentomics continues to improve the best validation score over time consistently. We observed that test scores also improved over time, however, Drug Discovery runs stagnated their normalized test scores around the 3 h mark, pointing to a phenomenon where the best test scores were reached in iterations with the non-best validation scores, and therefore not being selected as output iterations.

We do not observe a substantial dip in the normalized test score over time for any group, implying that on average the test scores were not negatively influenced by continuing the run past the point when test score stopped improving (e.g. due to overfitting to the validation set).

3.5 Agentomics explores a wide variety of strategies

To investigate the diversity of solutions explored during Agentomics runs, we analyzed the architectures implemented during the best-of-3 run for each of the 20 benchmarked datasets and plot their distribution in all iterations ($N = 364$, red bars) and just the output iterations ($N = 20$, blue bars) (Fig. 3). We observe that Agentomics explores a wide variety of strategies across its runs, implementing both traditional architectures like linear regression or support vector machines, as well as deep learning architectures like transformers and convolutional neural networks. The most common approach was to use ensemble methods like XGBoost, random forests, and custom ensembles composed of various individual architectures. We also observe that Agentomics commonly utilizes foundation model embeddings, using them in 20% of output iterations. These placed higher on the leaderboards than other runs (Mann–Whitney U-test, $P = 0.020$).

3.6 Analysis of weak runs

Based on the performance drop relative to the best run on each dataset, the 15% worst runs showed lower validation-test score correlations (median correlation 0.59 vs 0.97; Mann–Whitney U, $P = 0.037$). Additionally, in 6/7 datasets, they completed the fewest

Downloaded from https://academic.oup.com/bioinformatics/article/42/Supplement_1/btag250/8726289 by University of Malta user on 28 July 2026

successful iterations, suggesting they create unrepresentative validation sets or fail to complete enough iterations in the 8 h window.

Drug Discovery was the weakest domain overall for Agentomics runs. Their best foundation model-based iterations had lower normalized validation (median 0.27 vs 0.92; Mann–Whitney U, $P = 0.023$) and normalized test scores (median 0.28 vs 0.86; Mann–Whitney U, $P = 0.012$) compared to other domains. The validation-test score correlations were also lower in Drug Discovery runs compared to other domains (median = 0.907 vs 0.984; Mann–Whitney U, $P = 0.056$). This suggests that the available foundation models were weaker for Drug Discovery than for other domains, and that the created validation splits were less representative than in other domains. Notably, Drug Discovery runs also did not implement any graph neural networks, which are common in the Drug Discovery leaderboard models.

3.7 Case study: AGO2 CLASH Hejret 2023

We further analyze how Agentomics tackles the AGO2-CLASH Hejret 2023 benchmark, comprising 8193 miRNA–target chimeric pairs identified via AGO2 CLASH technology (Hejret *et al.* 2023). Each sample consists of two RNA sequences, and the task is to decide whether they bind or not in the context of the AGO2 protein. This dataset has been extensively studied using a range of machine learning architectures (Sammut *et al.* 2025). As a case study, we investigate the best-performing 8-h Agentomics run, which outperformed the test score of the human state-of-the-art (AUPRC 0.880 vs 0.860).

The run began with a logistic regression baseline on one-hot encoded RNA sequences, followed by gradient-boosted trees, and standard deep learning approaches including CNNs and transformers. The following iterations implemented interaction matrices encoding Watson-Crick and wobble base pairing, a representation closely aligned with known miRNA-target biology and similar to that used in expert-designed models. Notably, these representations and models were not provided to the agent. Agentomics further fine-tuned a foundation model (Nucleotide Transformer ~50M parameters; Dalla-Torre *et al.* 2025), experimented with cross-attention dual encoders, and implemented CNN ensembles of residual, depthwise-separable, and ConvNeXt-style (Liu *et al.* 2022) architectures. Multiple ensemble strategies were explored, including weighted averaging, stacking with logistic regression and XGBoost, and large ensembles of prior high-performing solutions. The output solution was a probability-weighted combination of five previously implemented ensembles, explored in five different iterations, and achieved the strongest overall validation set performance.

Overall, this case study demonstrates that Agentomics iteratively explores, refines, and recombines a wide range of modeling strategies, ultimately exceeding expert-designed solutions through extensive exploration of the strategy space.

3.8 Agentomics is reliable, fast, and cost effective

LLM-based systems need LLM completions to generate tool calls and responses. All Agentomics experiments were run with the recommended GPT-5.1-Codex-Max backbone, which currently (January 2026) bills at 1.25$/M input tokens and 10$/M output tokens. We have tracked the spending of each of Agentomics runs, and find that 8-h runs on average spend 9.4 ± 5.0 USD. The average cost of a single iteration is 0.45 ± 0.09 USD. The average duration for a single iteration is roughly 30 min, at 0.51 ± 0.32 h. Over a total of 60 runs, Agentomics generated a working reusable solution every time, achieving a success rate of 100%.

3.9 Agentomics is publicly available and open-source

We provide a user-friendly way to use Agentomics through a single command that will install necessary dependencies and guide user through configuring a run. Agentomics is open-source and can be used with a wide range of datasets and without requiring the user to have biomedical domain knowledge, programming experience, or machine learning expertise. Agentomics allows the user to customize each run through various parameters like maximum duration and LLM backbone.

4 Discussion

Agentomics is an autonomous LLM Agent-based system for end-to-end biomedical ML experimentation that takes a dataset as input, iteratively designs and implements complete ML solutions, and returns validated and ready-to-use training and inference code as well as model artifacts. We evaluated Agentomics across three biomedical domains: Regulatory Genomics, Protein Engineering, and Drug Discovery, on a total of 20 established benchmark datasets. We found that Agentomics outperformed all other LLM Agents in all domains, while also surpassing the human-

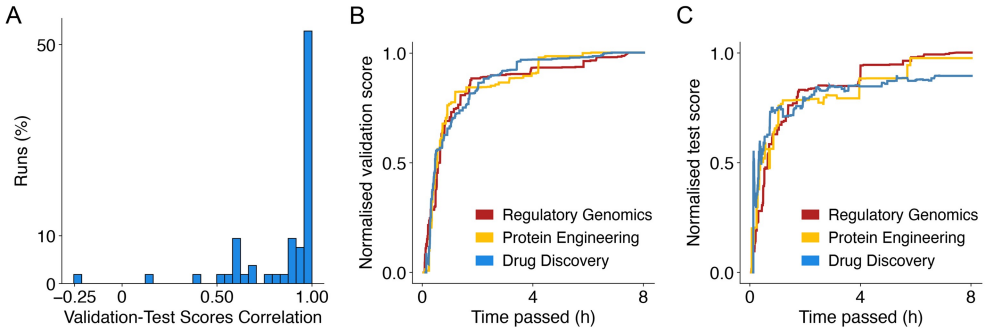


Figure 2 Agentomics indirectly optimizes for generalization. (A) Distribution of Validation-Test Scores Pearson Correlations from all runs. (B) Normalized best validation score and (C) Normalized test score (score on the test set if the run stopped at that time point) over 8 h averaged from all runs stratified by domain. Value of zero represents the worst model of each run and a value of one represents the best model based on validation and test scores, respectively.

Downloaded from https://academic.oup.com/bioinformatics/article/42/Supplement_1/btag250/8726289 by University of Malta user on 28 July 2026

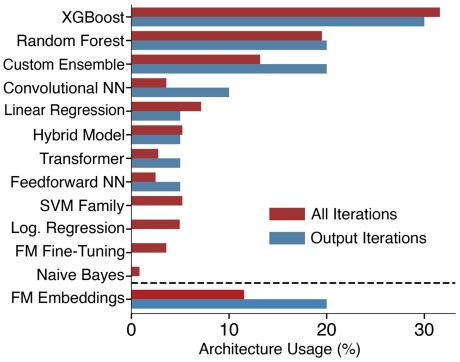


Figure 3 Agentomics implements varied strategies. Distribution of architecture types implemented by the best Agentomics run for each dataset, shown separately for all iterations ($N = 364$) and output (best validation score) iterations ($N = 20$). Additionally, a percentage of iterations that used foundation model embeddings is shown. Custom Ensemble architecture represents non-XGBoost, non-Random Forest, ensemble methods composed of various other architectures. Hybrid model architecture represents non-ensemble architectures that combine two or more simple architectures. FM=Foundation Model.

engineered state-of-the-art in 11/20 datasets. The gains were pronounced in Protein Engineering, where Agentomics exceeded human leaderboard performance in 6/6 tasks, and in Regulatory Genomics, where it improved on human state-of-the-art in 4/5 tasks, showing that the effect is not confined to a single modality or benchmark suite. These results provide evidence that fully autonomous agentic ML systems can now surpass human expert ML models at scale across multiple biomedical domains. This shifts the practical frontier from LLM-assisted development to autonomous discovery of expert-level models.

There are many ways to implement LLM Agent-based systems for ML. Differences in agent architecture, tool access, and evaluation design can influence the effectiveness of such systems. Agents operating without isolated environments are susceptible to unsafe code execution and test set label leakage, while unstructured agent architectures might hallucinate metrics and produce non-reusable outputs.

To guide the development of future LLM Agentic systems for robust ML, we would like to propose a set of standards: (i) runs should execute in a sandboxed environment (e.g. containerized with restricted mounts) to protect the user's machine and data; (ii) the test set must be strictly hidden from the agent to prevent data leakage; (iii) metrics should be computed by deterministic evaluation code, not by the agent; and (iv) to ensure reproducibility and transparency, the system should output reusable artifacts and scripts that can be inspected and used for training reproducibility and inference on arbitrary data.

Agentomics creates a validation set out of the original data and optimizes its performance over multiple iterations. We observe that optimizing this objective translates into improved generalization on the hidden test set in most cases, as evidenced by high validation-test scores correlation (Fig. 2A). We observe that this correlation is significantly lower for the overall weakest domain (Drug Discovery) and worst-performing runs, demonstrating the importance of using a representative validation set. We have also identified that Drug Discovery runs would benefit from stronger foundation models.

Examining the best runs across datasets shows that there is no single predominant architecture for biomedical ML. Strong solutions were discovered through iterative experimentation, spanned different model families, and often leveraged biomedical foundation models. This positions AutoML methods that only explore limited architecture and representation spaces as not optimal for biomedical ML.

A limitation of our study is that it does not explore long running (8+ hours) experiments. Based on our results, certain datasets, where metric curves did not plateau, would likely have benefited from longer experimentation. Additionally, our experiments are limited to a single LLM backbone, and used limited hardware resources. Access to more GPU resources would shorten model training times and allow experimentation with larger models and datasets within reasonable timeframes. Moreover, our results are limited to three replicates and demonstrate that a single 8-h run is not guaranteed to output the best-performing model. Finally, we do not isolate individual components of the Agentomics architecture to measure their contribution through ablation study.

Additionally, Agentomics currently only supports data in CSV format, limiting its use cases for modalities such as images or specialized data formats.

Although motivated by biomedical tasks, Agentomics implements supervised ML models in a generic way as a sequence of validated steps. Additionally, it is easily configurable and extendable with foundation models from any domain. This positions Agentomics as domain-agnostic and well-suited to be applied to diverse datasets.

5 Conclusions

Systems like Agentomics point to a near future where autonomous ML experimentation becomes routine. They are increasingly affordable to run, fast enough for practical use cases, and robust enough to produce reusable and reproducible artifacts. As these agentic systems become more capable and widely adopted, the key risk is not that they will fail to produce models, but that poorly designed implementations will flood the field with irreproducible code and inflated metrics due to test set leakage, ultimately eroding trust in computational biomedical ML. With Agentomics, we demonstrate that Agentic system for ML can be developed in a way that mitigates these risks, guaranteeing reliability, security, and standardized interfaces needed to develop high-quality scientific ML models at scale.

Author contributions

Vlastimil Martinek (Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Software, Validation, Visualization, Writing—original draft, Writing—review & editing), Andrea Gariboldi (Conceptualization, Investigation, Methodology, Software, Writing—original draft, Writing—review & editing), Dimosthenis Tzimotoudis (Conceptualization, Data curation, Investigation, Methodology, Software, Writing—original draft, Writing—review & editing), Mark Galea (Investigation, Methodology, Software, Writing—original draft, Writing—review & editing), Elissavet Zacharopoulou (Formal analysis, Investigation, Validation, Writing—original draft), Aitor Alberdi Escudero (Investigation,

Downloaded from https://academic.oup.com/bioinformatics/article/42/Supplement_1/btag250/8726289 by University of Malta user on 28 July 2026

Writing—original draft), Edward Blake (Investigation, Writing—original draft), David Čechák (Investigation, Resources, Writing—original draft), Luke Cassar (Investigation, Writing—original draft), Alessandro Balestrucci (Data curation, Investigation, Writing—original draft), and Panagiotis Alexiou (Conceptualization, Funding acquisition, Investigation, Methodology, Project administration, Resources, Supervision, Visualization, Writing—original draft, Writing—review & editing)

Conflict of interests

None declared.

Funding

V.M., A.G., D.T., A.B., E.Z., and P.A. were supported by HORIZON-WIDERA-2022 BioGeMT (ID: 101086768). E.B. and L.C. were supported by HORIZON-EIC-2022-Pathfinderchallenges-01-03 TargetMI (ID: 101114924). A.A.E., and P.A. were supported by University of Malta's Data Integrity and Stewardship Cluster (DISC). D.C. was supported by the Bioinformatics Core Facility of CEITEC Masaryk University via the National Center for Molecular Genomics (NCMG) Research Infrastructure (LM2023067 funded by Ministry of Education, Youth and Sports of the Czech Republic (MEYS CR)).

Data availability

Source code and documentation are freely available at: https://github.com/BioGeMT/Agentomics-ML.

References

Ahmad W, Simon E, Chithrananda S *et al.* ChemBERTa-2: towards chemical foundation models. arXiv, 2209.01712, 2022, preprint: not peer reviewed.

Ash JR, Wognum C, Rodríguez-Pérez R *et al.* Practically significant method comparison protocols for machine learning in small molecule drug discovery. *J Chem Inf Model* 2025;**65**:9398–411.

Azevedo K, Quaranta L, Calefato F *et al.* A multivocal literature review on the benefits and limitations of automated machine learning tools. arXiv, 2401.11366, 2024, preprint: not peer reviewed.

Chan JS, Chowdhury N, Jaffe O *et al.* MLE-bench: evaluating machine learning agents on machine learning engineering. arXiv, 2410.07095, 2024, preprint: not peer reviewed.

Dalla-Torre H, Gonzalez L, Mendoza-Revilla J *et al.* Nucleotide transformer: building and evaluating robust foundation models for human genomics. *Nat Methods* 2025;**22**:287–97.

Erickson N, Mueller J, Shirkov A *et al.* AutoGluon-tabular: robust and accurate AutoML for structured data. arXiv, 2003.06505, 2020, preprint: not peer reviewed.

Feurer M, Klein A, Eggensperger K *et al.* Efficient and robust automated machine learning. *Adv Neural Inf Process Syst* 2015;**23**:1–61.

Grešová K, Martinek V, Čechák D *et al.* Genomic benchmarks: a collection of datasets for genomic sequence classification. *BMC Genom Data* 2023;**24**:25.

Hejret V, Varadarajan NM, Klimentova E *et al.* Analysis of chimeric reads characterises the diverse targetome of AGO2-mediated regulation. *Sci Rep* 2023;**13**:22895.

Huang K, Zhang S, Wang H *et al.* Biomni: a general-purpose biomedical AI agent. bioRxiv, 2025.05.30.656746, 2025, preprint: not peer reviewed.

Huang Q, Vora J, Liang P *et al.* MLAgentBench: evaluating language agents on machine learning experimentation. arXiv, 2310.03302, 2024, preprint: not peer reviewed.

Jiang Z, Schmidt D, Srikanth D *et al.* AIDE: AI-driven exploration in the space of code. arXiv, 2502.13138, 2025, preprint: not peer reviewed.

Jimenez CE, Yang J, Wettig A *et al.* SWE-bench: can language models resolve real-world GitHub issues? In: *International Conference on Learning Representations*, Vol. 2024, pp. 54107–57, 2024.

Jin R, Zhang Z, Wang M *et al.* STELLA: self-evolving LLM agent for biomedical research. *arXiv:2507.02004,* 2025, preprint: not peer reviewed.

LeDell E, Poirier S *et al.* H2o AutoML: scalable automatic machine learning. In: *Proceedings of the AutoML Workshop at ICML*, Vol. 2020. p. 24. 2020.

Lin Z, Akin H, Rao R *et al.* Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science* 2023;**379**:1123–30.

Liu Z, Mao H, Wu C-Y *et al.* A ConvNet for the 2020s. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 11976–86, 2022.

Marx V. The big challenges of big data. *Nature* 2013;**498**:255–60.

Miller HE, Greenig M, Tenmann B *et al.* BioML-bench: evaluation of AI agents for end-to-end biomedical ML. 2025.09.01.673319 bioRxiv, 2025-09, 2025. preprint: not peer reviewed.

Nguyen E, Poli M, Faizi M *et al.* HyenaDNA: long-range genomic sequence modeling at single nucleotide resolution. *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, pp. 43177–201, 2023.

Notin P, Kollasch A, Ritter D *et al.* ProteinGym: large-scale benchmarks for protein fitness prediction and design. *Adv Neural Inf Process Syst* 2023;**36**:64331–79.

Penić RJ, Vlašić T, Huber RG *et al.* RiNALMo: general-purpose RNA language models can generalize well on structure prediction tasks. *Nat Commun* 2025;**16**:5671.

Qiao L, Ye P, Ren Y *et al.* Model decides how to tokenize: adaptive DNA sequence tokenization with MxDNA. *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, pp. 66080–107, 2024.

Ross J, Belgodere B, Chenthamarakshan V *et al.* Large-scale chemical language representations capture molecular structure and properties. *Nat Mach Intell* 2022;**4**:1256–64.

Sammut S, Gresova K, Tzimotoudis D *et al.* miRBench: novel benchmark datasets for microRNA binding site prediction that mitigate against prevalent microRNA frequency class bias. *Bioinformatics* 2025;**41**:i542–51.

Shinn N, Cassano F, Gopinath A *et al.* Reflexion: language agents with verbal reinforcement learning. *Adv Neural Inf Process Syst* 2023;**36**:8634–52.

Stephens ZD, Lee SY, Faghri F *et al.* Big data: astronomical or genomical? *PLoS Biol* 2015;**13**:e1002195.

Downloaded from https://academic.oup.com/bioinformatics/article/42/Supplement_1/btag250/8726289 by University of Malta user on 28 July 2026

Thoutam V, Ellsworth D. MSAMamba: adapting subquadratic models to long-context DNA MSA analysis. In: *ICML 2024 Workshop on Theoretical Foundations of Foundation Models (TF2M 2024)*, 2024.

Wang L, Ma C, Feng X *et al.* A survey on large language model based autonomous agents. *Front Comput Sci* 2024;**18**:186345.

Wolf T, Debut L, Sanh V *et al.* Transformers: state-of-the-art natural language processing. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. 2020, 38–45.

Yao S, Zhao J, Yu D *et al.* ReAct: synergizing reasoning and acting in language models. In: *International Conference on Learning Representations (ICLR 2023),* 2023.

Yu T, Cheng L, Khalitov R *et al.* Self-distillation improves self-supervised learning for DNA sequence inference. *Neural Netw* 2025;**183**:106978.

Zurgham AM, Mehwish M. Optimization of deep learning convolutional neural network for genomic sequence classification. *Int J Adv Multidisc Res Stud* 2025;**5**:1159–62.

Downloaded from https://academic.oup.com/bioinformatics/article/42/Supplement_1/btag250/8726289 by University of Malta user on 28 July 2026

© The Author(s) 2026. Published by Oxford University Press.
This is an Open Access article distributed under the terms of the Creative Commons Attribution License (https://creativecommons.org/licenses/by/4.0/), which permits unrestricted reuse, distribution, and reproduction in any medium, provided the original work is properly cited.
Bioinformatics, 2026, 42, 1–10
https://doi.org/10.1093/bioinformatics/btag250
ISMB proceedings Supplement